Федеральное государственное автономное образовательное учреждение высшего образования
“Национальный исследовательский университет ИТМО”
Факультет программной инженерии и компьютерных технологий

Лабораторная работа №1

По дисциплине «Бизнес-логика программных систем»

Вариант 666

Выполнили: Соколова Полина Дмитриевна
Кулешова Екатерина Михайловна
Группа: P3315
Преподаватель: Шибаев Семен Сергеевич

Санкт-Петербург
2026

Задание:

Лабораторная работа #1

Введите вариант 666

Вариант №666: МТС — связь и экосистема цифровых сервисов — https://mts.ru. Бизнес-процесс: работа с разделами основного сайта — мобильная связь (в т.ч. пополнение счёта), для дома, сервисы и подписки, финансы.

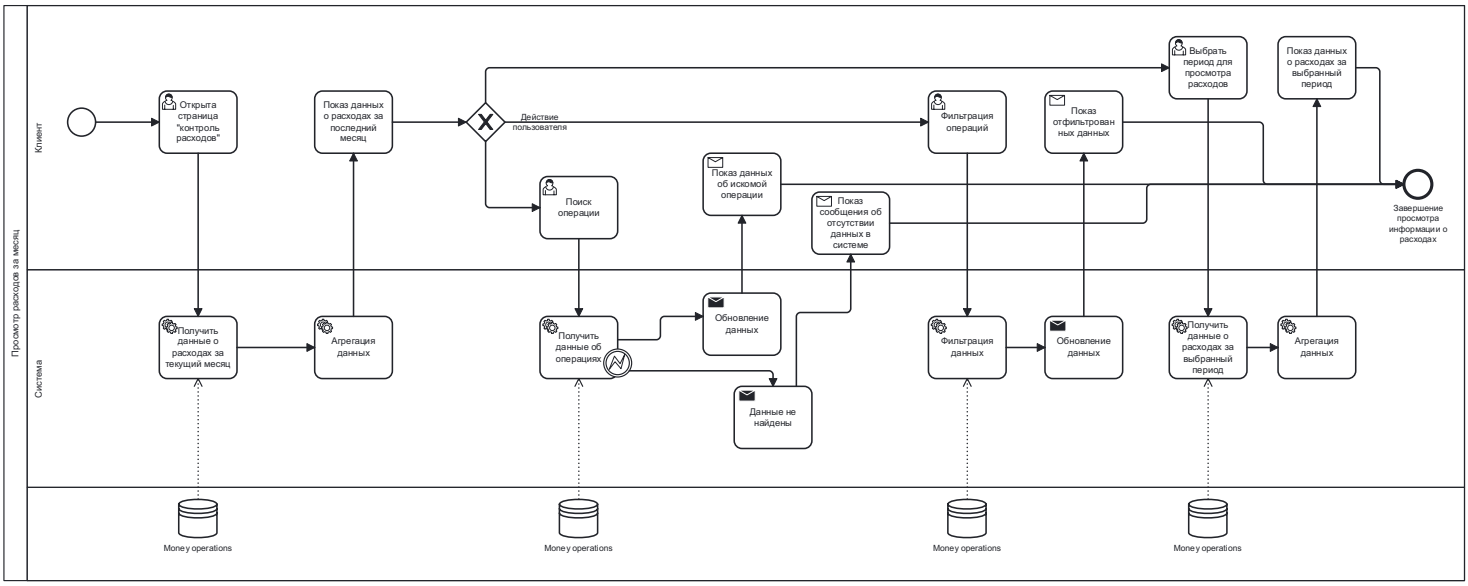
Описать бизнес-процесс в соответствии с нотацией BPMN 2.0, после чего реализовать его в виде приложения на базе Spring Boot.

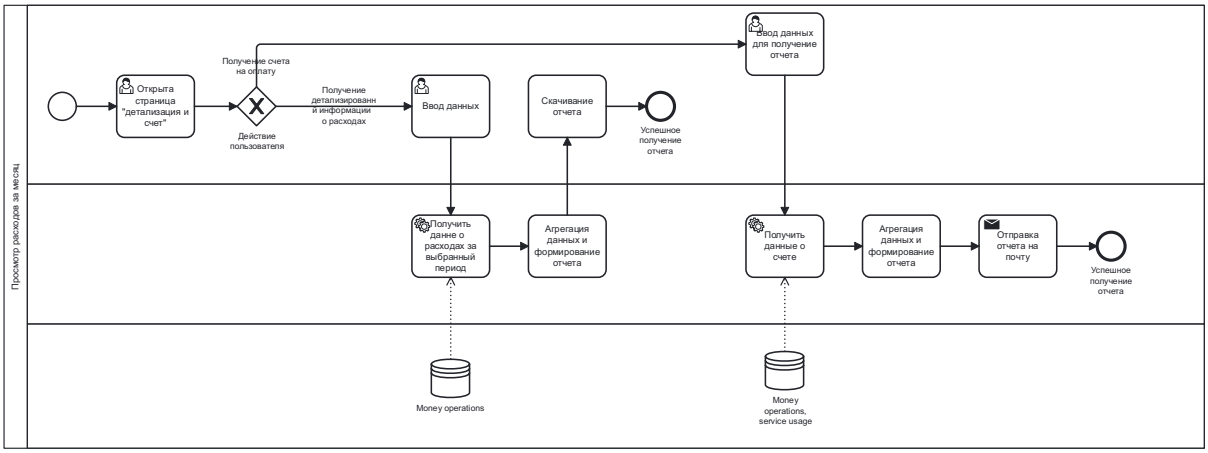
Порядок выполнения работы:

1. Выбрать один из бизнес-процессов, реализуемых сайтом из варианта задания.
2. Утвердить выбранный бизнес-процесс у преподавателя.
3. Специфицировать модель реализуемого бизнес-процесса в соответствии с требованиями BPMN 2.0.
4. Разработать приложение на базе Spring Boot, реализующее описанный на предыдущем шаге бизнес-процесс. Приложение должно использовать СУБД PostgreSQL для хранения данных, для всех публичных интерфейсов должны быть разработаны REST API.
5. Разработать набор curl-скриптов, либо набор запросов для REST клиента Insomnia для тестирования публичных интерфейсов разработанного программного модуля. Запросы Insomnia оформить в виде файла экспорта.
6. Развернуть разработанное приложение на сервере `helios`.

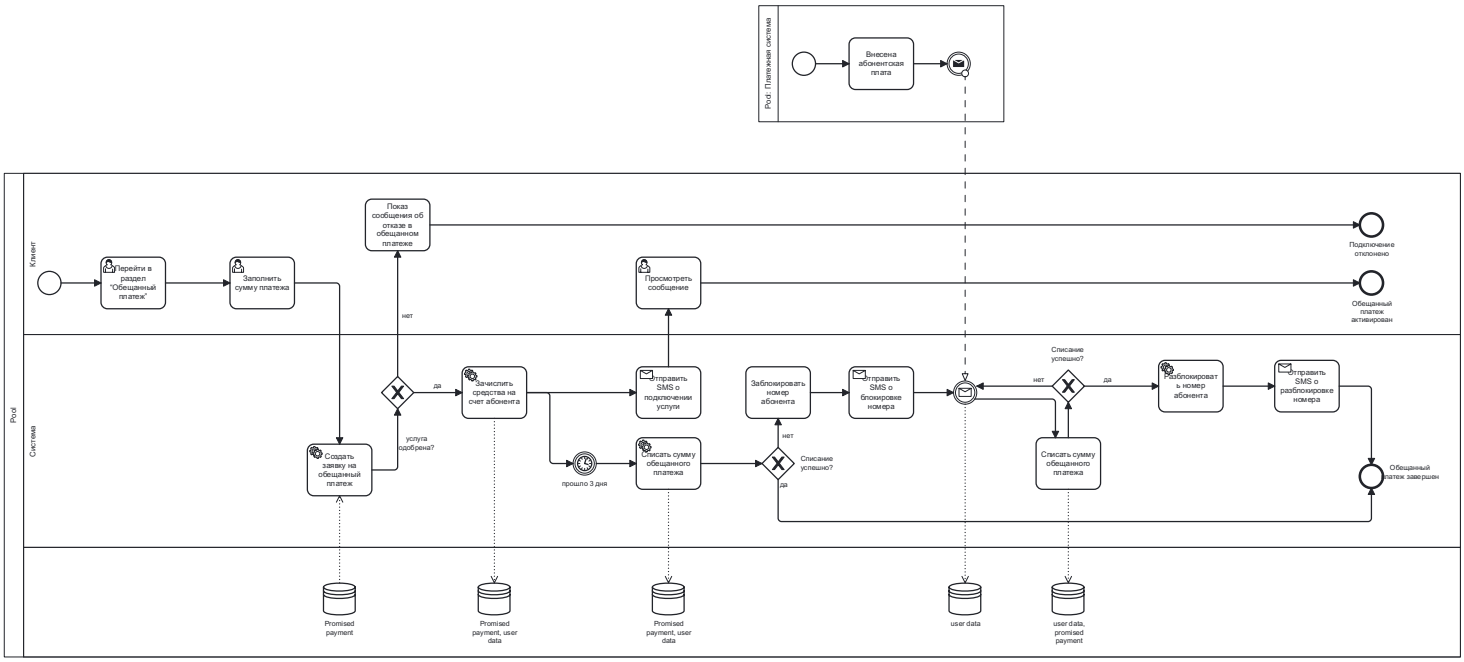
Выполнение:

1. **Модель потока управления для автоматизируемого бизнес-процесса №1: Просмотр информации о расходах в течение заданного периода**

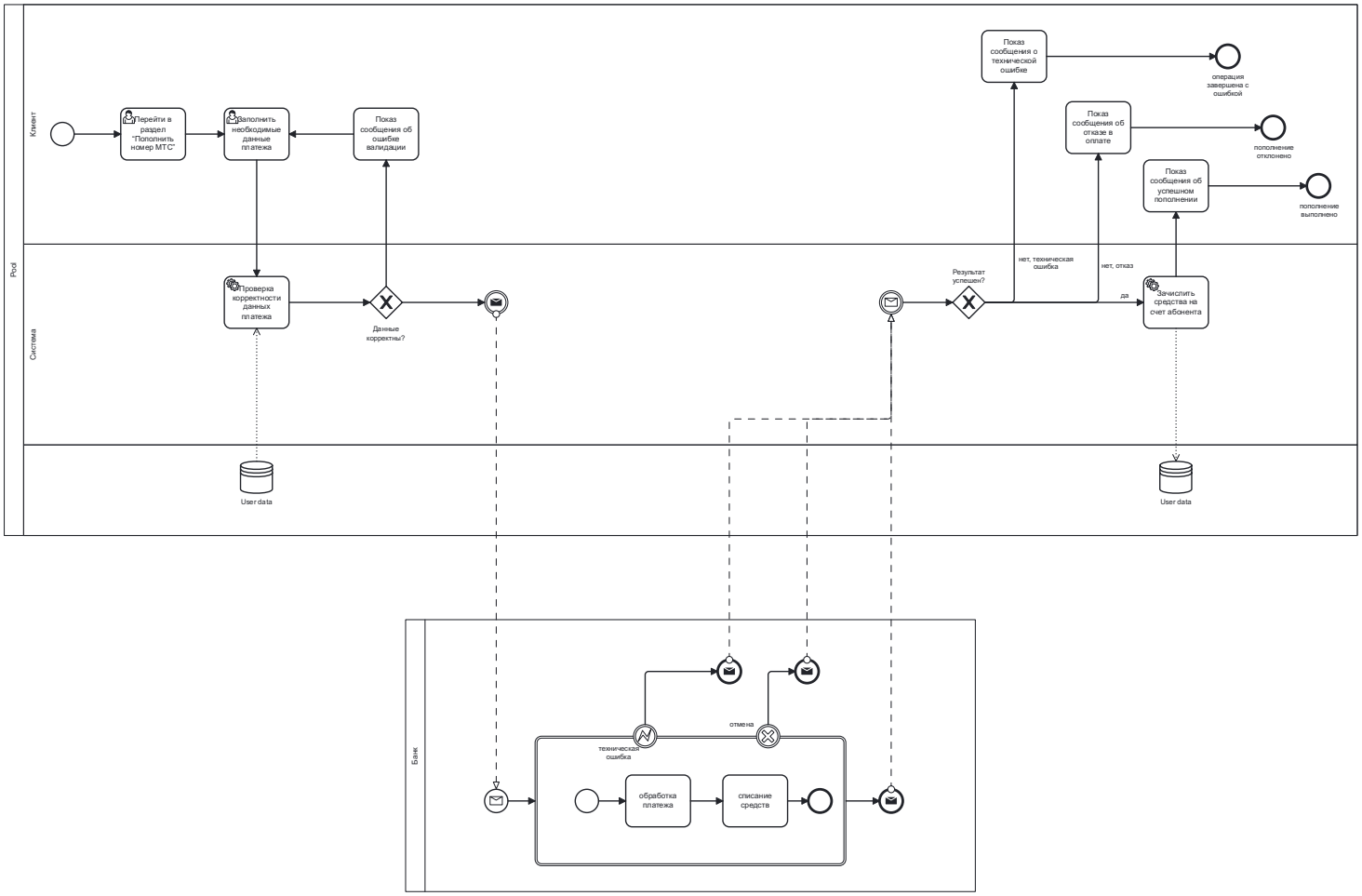




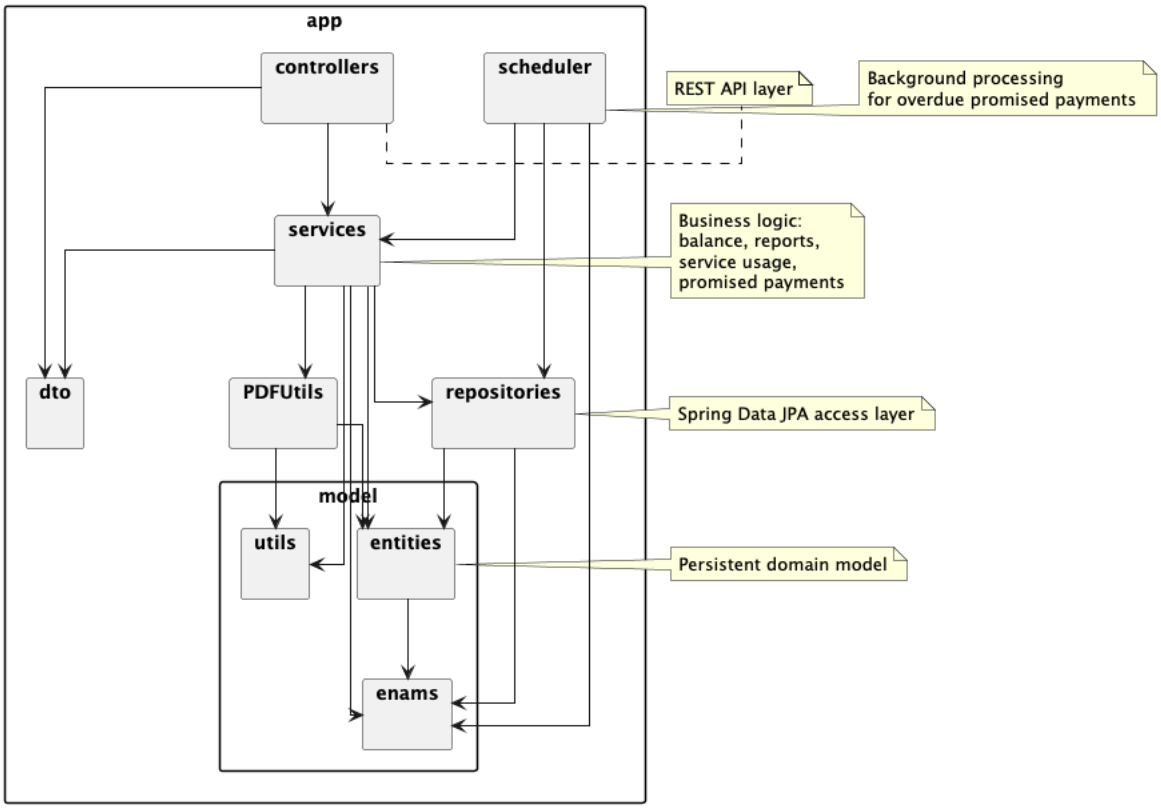
2. Модель потока управления для автоматизируемого бизнес-процесса №2: Оформление услуги обещанного платежа



3. Модель потока управления для автоматизируемого бизнес-процесса №3: Пополнение лицевого счета абонента



4. **UML-диаграмма пакетов разработанного приложения**



5. **Спецификация REST API для всех публичных интерфейсов разработанного приложения**

Разработанное приложение включает в себя публичные интерфейсы, относящиеся к следующим функциональным подсистемам:

1. Управление балансом абонента
2. Подключение и обработка обещанных платежей
3. Получение информации о расходах
4. Формирование и отправка отчетов
5. Регистрация использования сервисов связи
6. Управление пользователями
7. Управление данными абонентов

Базовый префикс всех REST-методов /api

Общие правила взаимодействия:

1. Для методов POST тело запроса передается в формате JSON.
2. Для методов GET входные данные передаются либо через path parameter, либо через query parameter.
3. Даты передаются в формате ISO yyyy-MM-dd.
4. Значения LocalDateTime в ответах возвращаются в стандартном формате ISO date-time.

В приложении используются:

1. централизованный обработчик исключений через @RestControllerAdvice;
2. локальная обработка RuntimeException внутри некоторых контроллеров.

Тело ошибки в разных методах:

1. либо строка;
2. либо JSON-объект вида

```
{
  "message": "Текст ошибки"
}
```

Коды ответа:

200 OK - операция выполнена успешно.

201 Created - объект успешно создан.

400 Bad Request - ошибка входных данных, бизнес-ограничение или некорректный запрос.

404 Not Found - запрашиваемый объект не найден.

500 External Server Error - внутренняя ошибка сервера.

Ниже собраны endpoint’ы с акцентом именно на тело запроса и возможные ответы.

Подсистема управления балансом

1. `GET /api/balance/{userDataId}`

Назначение: получение текущего баланса пользователя.

Тело запроса:

- отсутствует

Возможные ответы:

- 200 OK

json

```
{
  "userDataId": 1,
  "accountNumber": "ACC1000001",
  "balance": 1200.00
}
```

- 400 Bad Request

json

```
"Пользователь не найден"
```

2. `POST /api/balance/top-up`

Назначение: пополнение баланса пользователя.

Тело запроса:

json

```
{
  "cardNumber": "1111222233334444",
  "cvc": "123",
```

```
  "amount": 500.00,
  "userDataId": 1
}
```

Возможные ответы:

- 200 OK

json

```
"Баланс успешно пополнен"
```

- 400 Bad Request

json

```
"Банк отклонил операцию (проверьте сумму и данные)"
```

- 400 Bad Request

json

```
"Пользователь не найден"
```

- 500 Internal Server Error

json

```
"Техническая ошибка банка"
```

3. `POST /api/balance/spend`

Назначение: списание средств с баланса.

Тело запроса:

```
json
{
  "amount": 250.00,
  "userDataId": 1,
  "name": "Абонентская плата"
}
```

Возможные ответы:

- 200 OK

json

```
"Баланс успешно уменьшен"
```

- 400 Bad Request

json

```
"Недостаточно средств"
```

- 400 Bad Request

json

```
"Пользователь не найден"
```

Подсистема обещанного платежа

4. `POST /api/promised-payment/take`

Назначение: подключение обещанного платежа.

Тело запроса:

json

```
{
  "userDataId": 1,
  "amount": 1000.00
}
```

Возможные ответы:

- 200 OK

```
json
"Обещанный платеж успешно подключен"
```

- 400 Bad Request

```
json
"Сумма обещанного платежа должна быть положительной"
```

- 400 Bad Request

```
json
"За один раз можно взять не более 1500 руб."
```

- 400 Bad Request

```
json
"Превышен месячный лимит обещанных платежей. Доступно: 500 руб."
```

- 400 Bad Request

```
json
"Нельзя взять новый обещанный платеж, пока не погашена просроченная задолженность"
```

- 400 Bad Request

```
json
"Пользователь не найден"
```

Подсистема учета расходов

5. `GET /api/expenses/forPeriod`

Назначение: получение расходов за период.

Тело запроса:

- отсутствует

Возможные ответы:

- 200 OK

```
json
  {
```

```
    "accountNumber": "ACC1000001",
    "moneyOperationTime": "2026-03-20T10:15:00",
    "moneyOperationName": "Абонентская плата",
    "moneyOperationType": "EXPENSE",
    "moneyOperationAmount": 300.00
  }
```

- 404 Not Found

json

```
{
  "message": "Пользователь с заданным лицевым счетом не найден"
}
```

- 500 Internal Server Error

json

```
{
  "message": "Внутренняя ошибка сервера"
}
```

6. `GET /api/expenses/forPeriodAndOperationType`

Назначение: получение расходов за период по типу операции.

Тело запроса:

- отсутствует

Возможные ответы:

- 200 OK

json

```
{
    "accountNumber": "ACC1000001",
    "moneyOperationTime": "2026-03-20T10:15:00",
    "moneyOperationName": "Регистрация платежа",
    "moneyOperationType": "INCOME",
    "moneyOperationAmount": 500.00
  }
```

- 404 Not Found

json

```
{
  "message": "Пользователь с заданным лицевым счетом не найден"
}
```

- 500 Internal Server Error

json

```
{
  "message": "Внутренняя ошибка сервера"
}
```

7. `GET /api/expenses/forPeriodAndOperationName`

Назначение: получение расходов за период по названию операции.

Тело запроса:

- отсутствует

Возможные ответы:

- 200 OK

json

```
  {
    "accountNumber": "ACC1000001",
    "moneyOperationTime": "2026-03-20T10:15:00",
    "moneyOperationName": "Абонентская плата",
    "moneyOperationType": "EXPENSE",
    "moneyOperationAmount": 300.00
  }
```

- 404 Not Found

json

```
{
  "message": "Пользователь с заданным лицевым счетом не найден"
}
```

- 404 Not Found

json

```
{
  "message": "Операция платеж не найдена"
}
```

- 500 Internal Server Error

json

```
{
  "message": "Внутренняя ошибка сервера"
}
```

Подсистема формирования отчетов

8. `GET /api/report/getInformationAboutExpenses`

Назначение: формирование PDF-отчета по расходам.

Тело запроса:

- отсутствует

Возможные ответы:

- 200 OK
- тело ответа: PDF-файл (application/pdf)
- 404 Not Found

json

```
{
  "message": "Пользователь с заданным лицевым счетом не найден"
}
```

- 400 Bad Request

json

```
{
  "message": "Детализированная информация доступна только за время до
текущего момента"
}
```

- 500 Internal Server Error

json

```
{
  "message": "Ошибка при отправке отчета на почту"
}
```

- 500 Internal Server Error

json

```
{
  "message": "Внутренняя ошибка сервера"
}
```

9. `GET /api/report/getInformationAboutExpensesOnEmail`

Назначение: формирование отчета по расходам и отправка на email.

Тело запроса:

- отсутствует

Возможные ответы:

- 200 OK

json

```
{
  "message": "Отчет успешно сформирован и отправлен на почту
test@example.com"
```

```
}
```

- 400 Bad Request

json

```
{
  "message": "must be a well-formed email address"
}
```

- 404 Not Found

json

```
{
  "message": "Пользователь с заданным лицевым счетом не найден"
}
```

- 500 Internal Server Error

json

```
{
  "message": "Ошибка при отправке отчета на почту"
}
```

- 500 Internal Server Error

json

```
{
  "message": "Внутренняя ошибка сервера"
}
```

10. `GET /api/report/getBill`

Назначение: формирование счета и отправка на email.

Тело запроса:

- отсутствует

Возможные ответы:

- 200 OK

json

```
{
  "message": "Счет успешно сформирован и отправлен на почту
test@example.com"
}
```

- 400 Bad Request

json

```
{
  "message": "must be a well-formed email address"
}
```

- 404 Not Found

json

```
{
  "message": "Пользователь с заданным лицевым счетом не найден"
}
```

- 500 Internal Server Error

json

```
{
  "message": "Ошибка при отправке отчета на почту"
}
```

- 500 Internal Server Error

json

```
{
  "message": "Внутренняя ошибка сервера"
}
```

Подсистема регистрации использования сервисов

11. `POST /api/serviceUsage/createServiceUsage`

Назначение: регистрация использования сервиса связи.

Тело запроса:

json

```
{
  "operationType": "CALL",
  "direction": "OUTGOING",
  "name": "Исходящий звонок",
  "unitsUsed": 35,
  "operationTime": "2026-03-20T12:30:00",
  "userData": {
    "accountNumber": "ACC1000001"
  }
}
```

Возможные ответы:

- 201 Created

json

```
{
  "message": "Использование сервисов МТС успешно зарегистрировано"
}
```

- 404 Not Found

json

```
{
  "message": "Пользователь с заданным лицевым счетом не найден"
}
```

- 400 Bad Request

json

```
{
  "message": "У пользователя недостаточно минут для совершения звонка"
}
```

- 400 Bad Request

json

```
{
  "message": "У пользователя недостаточно слов для отправки сообщения"
}
```

- 400 Bad Request

json

```
{
  "message": "У пользователя недостаточно интернета"
}
```

- 500 Internal Server Error

```
json
{
  "message": "Внутренняя ошибка сервера"
}
```

Подсистема управления пользователями

12. `POST /api/user/createUser`

Назначение: создание пользователя.

Тело запроса:

json

```
{
  "lastName": "Иванов",
  "firstName": "Иван",
  "middleName": "Иванович",
  "birthDate": "1999-05-14",
  "passportSeries": "1234",
  "passportNumber": "567890"
}
```

Возможные ответы:

- 201 Created

json

```
{
  "message": "Пользователь успешно создан"
}
```

- 500 Internal Server Error

json

```
{
  "message": "Внутренняя ошибка сервера"
}
```

13. `GET /api/user`

Назначение: получение списка пользователей.

Тело запроса:

- отсутствует

Возможные ответы:

- 200 OK

json

```
  {
    "id": 1,
    "lastName": "Иванов",
    "firstName": "Иван",
    "middleName": "Иванович",
    "birthDate": "1999-05-14",
    "passportSeries": "1234",
    "passportNumber": "567890"
  }
```

- 500 Internal Server Error

json

```
{
  "message": "Внутренняя ошибка сервера"
}
```

Подсистема управления данными абонентов

14. `POST /api/userData/createUserData`

Назначение: создание данных абонента.

Тело запроса:

json

```
{
  "accountNumber": "ACC1000001",
  "phoneNumber": "79161234567",
  "balance": 1200.00,
  "remainingSeconds": 500,
  "remainingBytes": 10737418240,
  "remainingSms": 100,
  "isBlocked": false,
  "user": {
    "id": 1
  }
}
```

Возможные ответы:

- 201 Created

json

```
{
  "message": "Данные пользователя добавлены"
}
```

- 400 Bad Request

json

```
{
  "message": "Пользователя с заданным id не существует"
}
```

- 500 Internal Server Error

json

```
{
  "message": "Внутренняя ошибка сервера"
}
```

15. `POST /api/userData/increaseRemaining`

Назначение: увеличение остатков минут, SMS и интернет-трафика.

Тело запроса:

json

```
{
  "accountNumber": "ACC1000001",
  "remainingSeconds": 300,
  "remainingSms": 50,
```

```
  "remainingBytes": 1024
}
```

Возможные ответы:

- 200 OK

json

```
{
  "message": "Данные пользователя добавлены"
}
```

- 404 Not Found

json

```
{
  "message": "Пользователь с заданным лицевым счетом не найден"
}
```

- 500 Internal Server Error

json

```
{
  "message": "Внутренняя ошибка сервера"
}
```

16. `GET /api/userData`

Назначение: получение списка данных абонентов.

Тело запроса:

- отсутствует

Возможные ответы:

- 200 OK

json

```
  {
    "id": 1,
    "accountNumber": "ACC1000001",
    "phoneNumber": "79161234567",
    "balance": 1200.00,
    "remainingSeconds": 500,
    "remainingBytes": 10737418240,
    "remainingSms": 100,
    "isBlocked": false
  }
```

- 500 Internal Server Error

json

```
{
  "message": "Внутренняя ошибка сервера"
}
```

5. **Ссылка на репозиторий с исходным кодом**

   https://github.com/pollee343/BLPS

6. **Вывод**

   В ходе выполнения работы было разработано серверное приложение, реализующее REST API для управления пользователями, данными абонентов, балансом, расходами, использованием сервисов связи, отчетами и обещанными платежами. В процессе разработки была спроектирована многослойная архитектура приложения, включающая контроллеры, сервисы, репозитории и сущности предметной области.

   Таким образом, поставленная цель работы была достигнута: разработано функционирующее серверное приложение с корректно реализованной бизнес-логикой, формализованной структурой, описанными интерфейсами и подготовленной проектной документацией.